

Hardware Bring-Up & Bench Data Sheet (v2)

3D Ultrasonic Projectile Landing-Prediction System - Politecnico di Milano, Measurements for Mechanical Engineering, A.Y. 2025-26

v2 (2026-07-03), supersedes BENCH_PROCEDURE.pdf. Updated for the post-audit-remediation code (233 tests): the G1 multilateration ground-truth prompt (centroid + 2 recommended sensors; the old two-tape L_centroid/L_S1 prompt no longer exists), simulate_session.py processing by default (the old --plot flag was REMOVED; v1's Phase-5 command now errors), automatic per-throw report.html + campaign.html rendering after every terminal throw, and a clean no-crash failure path. Use this when the Arduino Uno + 3x HC-SR04 + breadboard arrive. Work top to bottom; do not skip a gate. All commands run from the project root. Replace COM5 with your real port everywhere.

STANDING RULES

Always .\env\Scripts\python.exe ... - never bare python / pytest.

Pass --port COMx on every script (config default /dev/ttyACM0 is Linux).

Pass the real --temperature-c on every characterization run (it sets the speed of sound).

Do not guess kalman.sigma_pos_m or simulator.beam_half_angle_deg - they come from Phase 4.

Raw logs are sacred (CLAUDE.md 0.4) - never delete a captured raw_serial.log. run_session now REFUSES to reuse a session directory that already holds one.

If a gate FAILS, the number is real - fix the cause, never widen the tolerance.

Session date	Operator	Ambient T [C]	Port
_____	_____	_____	COM_____

PHASE 0 - Toolchain & first connection

#	Action	Done
0.1	Install Arduino IDE (or arduino-cli). None is installed on the dev machine (Part 3 S-1).	☐
0.2	Plug in the Uno; let Windows install the USB-serial driver.	☐
0.3	Find the COM port: Device Manager -> Ports (COM & LPT). Record it above.	☐
0.4	Compile-check every sketch before flashing (arduino:avr:uno). No sketch was ever toolchain-verified.	☐

PHASE 1 - Wiring (USB unplugged while wiring)

Pin map per CLAUDE.md sec. 3 - **do not deviate**; a swapped sensor silently corrupts trilateration.

Sensor	VCC	GND	TRIG	ECHO
S1	+ rail	- rail	D2	D3
S2	+ rail	- rail	D4	D5
S3	+ rail	- rail	D6	D7

Arduino 5V -> + rail, GND -> - rail. 5 V logic, direct connections, **no voltage dividers**. Sketch 01 needs only S1; add S2 + S3 before sketch 03. Twist/bundle each sensor's 4 leads; **label bundles S1/S2/S3 at both ends**; tug-test every jumper; confirm common ground.

#	Check	Done
1.1	All three VCC on +, all three GND on -, common ground confirmed	☐
1.2	TRIG/ECHO match the table exactly; bundles labelled both ends	☐
1.3	Every jumper tug-tested	☐

PHASE 2 - Firmware gates (sketches 01 -> 05, in order)

Flash each sketch, hand-check its in-file criterion, tick the box, **then** move to the next.

Sketch 01 - 01_single_sensor.ino (S1 only)

Fires S1 every 100 ms, prints echo_us / display-only distance. **Criterion:** flat wall at tape-measured distance reads **within +/-1 cm** (validates correctness: round-trip /2, constants, units).

Tape distance	Reading on screen	Within +/-1 cm?
0.50 m	_____ m	☐
1.00 m	_____ m	☐
1.50 m	_____ m	☐

Sketch 01 PASS ☐

Sketch 02 - 02_timeout_discipline.ino (S1 only)

Adds the 18 ms absolute-scheduler slot + 12500 us timeout. Prints echo / dt / cyc. (a) forced timeouts print literal 0; (b) dt ~ 18000 us every line, us-level jitter, **no drift** over ~1000 cycles.

Check	Observed	OK?
Timeout prints 0	_____	[]
Typical dt	_____ us	[]
Drift after ~1000 cycles (= start + 1000 x 18000)	_____	[]

Sketch 02 PASS []

Sketch 03 - 03_three_sequential.ino (all three)

S1 at t, S2 at t+18 ms, S3 at t+36 ms (period 54 ms). **Criterion:** readings change independently per sensor; firing order verified by **blocking each sensor in turn** - your real wiring-identity check.

Block this sensor	Only this reading changes?
S1	[]
S2	[]
S3	[]

Sketch 03 PASS [] labels confirmed map to correct pods []

Sketch 04 - 04_serial_csv.ino

Machine CSV: header # fw=04, slot_ms=18, timeout_us=12500, then sensor_id,echo_us,timestamp_us. **Criterion:** Python parses 1000 lines, **zero malformed**. Capture to a file, then dry-run in replay mode:

```
.\env\Scripts\python.exe scripts\verify_timing.py --replay <captured_log> --min-lines 1000
```

Malformed rows: _____ Sketch 04 PASS []

Sketch 05 - 05_timing_verify.ino (flight firmware)

Identical to 04 except header fw=05. Automated PC-side test; exit 0 iff all pass. **Watch worst-deviation us** - the simulator gave 0 us; hardware jitter must stay well inside +/-50 us. If not, it's real - surface it, don't widen.

```
.\env\Scripts\python.exe scripts\verify_timing.py --port COM5
```

#	Check	Worst dev	PASS?
1	per-sensor period = 54.000 ms +/- 50 us	_____ us	[]
2	S2-S1 = 18 ms, S3-S1 = 36 ms +/- 50 us	_____ us	[]
3	timestamps strictly monotonic after unwrap	-	[]
4	zero parse errors	_____	[]

Sketch 05 PASS (exit 0) [] -> Firmware gate (sec. 10 item 12) COMPLETE

PHASE 3 - Background-only sanity check

Confirms calibration, band enable/disable, and (if ceiling_height_m is set in config.yaml) the A2 wrap-around ghost. Clear the volume, then:

```
.\env\Scripts\python.exe scripts\run_session.py --background-only --port COM5
```

Per sensor	mu_bg [m]	sigma_bg [mm]	band w [mm]	enabled?
S1	_____	_____	_____	[]
S2	_____	_____	_____	[]
S3	_____	_____	_____	[]

Open-air -> bands typically **disabled** (correct). Loud overlap/ghost warning? _____ If a static reflector lands inside [0.5 m, d_max], **the room must change, not the thresholds**. Phase 3 OK []

PHASE 4 - Static characterization <- THE DATA SHEET

Goal: produce the two config numbers. Tool: characterize_static.py (one report per run) -> analyze_static.py (aggregate). Record the **ambient temperature once** and use it in every command. T_ambient used for all Phase-4 runs: _____ C

4A - Flat board (proves electronics; clean offset + noise floor)

Mount a flat board perpendicular to the array at a tape-to-**face** distance.

```
.\env\Scripts\python.exe scripts\characterize_static.py --target board --reference-m 1.000 --temperature-c <T> --port COM5
```

Sensor	n	timeout %	mean [m]	sigma [mm]	offset [mm]
S1	_____	_____	_____	_____	_____
S2	_____	_____	_____	_____	_____

Sensor	n	timeout %	mean [m]	sigma [mm]	offset [mm]
S3	_____	_____	_____	_____	_____

4B - Ball distance series (gives kalman.sigma_pos_m)

Tennis ball on a stand; tape to the **nearest surface** (the tool adds the 0.0335 m radius for you). Run all three distances:

```
.\env\Scripts\python.exe scripts\characterize_static.py --target ball --reference-m 0.967 --temperature-c <T> --port COM5
... repeat with --reference-m 1.267 and --reference-m 1.667 ...
```

~1.0 m (tape _____ m)

Sensor	n	timeout %	mean [m]	sigma [mm]	offset [mm]
S1	_____	_____	_____	_____	_____
S2	_____	_____	_____	_____	_____
S3	_____	_____	_____	_____	_____

~1.3 m (tape _____ m)

Sensor	n	timeout %	mean [m]	sigma [mm]	offset [mm]
S1	_____	_____	_____	_____	_____
S2	_____	_____	_____	_____	_____
S3	_____	_____	_____	_____	_____

~1.7 m (tape _____ m)

Sensor	n	timeout %	mean [m]	sigma [mm]	offset [mm]
S1	_____	_____	_____	_____	_____
S2	_____	_____	_____	_____	_____
S3	_____	_____	_____	_____	_____

Detectability gate (A5): if the ball shows **persistent timeouts at 1.3-1.7 m**, do NOT swap targets silently - escalate a substitution (taped ball / hard sphere) to **Prof. Zappa**. It is an approval item, not a code change.

Derive kalman.sigma_pos_m: take a representative sigma across sensors at the realistic distances (median, or the larger/conservative end - sigma grows with range). -> **sigma_pos_m** = _____ m (in metres, e.g. 0.006). *Report note:* this is a range sigma used as a position-axis noise; horizontal position noise ~ sigma x GDOP (~2.3). State whether you applied the GDOP factor in the error budget.

4C - Ball angle sweep (gives simulator.beam_half_angle_deg)

Fixed distance (~1.0-1.3 m). **Sweep ONE sensor at a time;** --angle-deg is that sensor's off-axis angle (ball's displacement from the 55-deg-elevation inward aim axis). Keep widening until the timeout fraction passes **50%** (must bracket the crossing).

```
.\env\Scripts\python.exe scripts\characterize_static.py --target ball --reference-m 1.000 --angle-deg 0
--temperature-c <T> --port COM5
... repeat with --angle-deg 3 / 6 / 9 / 12 / 15 ...
```

Record the **swept sensor's** timeout % at each angle (other columns are incidental). Swept sensor: S_____ distance _____ m

Angle [deg]	0	3	6	9	12	15
timeout %	_____	_____	_____	_____	_____	_____

(Optional: repeat for the other two sensors.) Swept S_____: crossing ~ _____ deg Swept S_____: crossing ~ _____ deg

4D - Run the aggregator

```
.\env\Scripts\python.exe scripts\analyze_static.py data\sessions\*_static\static_report.json
```

Beam half-angle crossings: S1 _____ deg S2 _____ deg S3 _____ deg **Suggested beam_half_angle_deg:** _____ (median of bracketed sensors)

Sensor	offset [mm]	sigma/sqrt(N) [mm]	Verdict
S1	_____	_____	[] SIGNIFICANT [] within noise
S2	_____	_____	[] SIGNIFICANT [] within noise
S3	_____	_____	[] SIGNIFICANT [] within noise

PHASE 5 - Update config.yaml from the measurements (one edit)

Edit config.yaml with the Phase-4 numbers (and Phase-6 below). Do it once, deliberately. YAML trap: exponential floats need the decimal point / signed exponent (1.0e-4, 1.0e+4) or PyYAML parses them as strings.

Key	Old	New (measured)
kalman.sigma_pos_m	0.010	_____
simulator.beam_half_angle_deg	7.0	_____

If any sensor offset is **SIGNIFICANT** (4D): add the constant per-sensor offset subtraction at the **acquisition** layer (small edit to acquisition/run_session), deliberately **OUTSIDE bias.py** (sec. 7). This hook does not exist yet. Note: S1 ____ mm S2 ____ mm S3 ____ mm.

Cone re-check after tuning - regenerate synthetic throws *without* --beam-cone off and confirm ~5-6 triplets ($\geq \text{min_fit_points}$). **v2 note: processing/plots are now the DEFAULT - there is NO --plot flag any more** (v1's command errors on the current code; --no-plot exists to skip processing):

```
.\env\Scripts\python.exe scripts\simulate_session.py --n-sessions 3 --seed 11
```

If still 0 triplets at campaign speed, the cone is genuinely too narrow -> real detectability finding (A5), not a typo. Triplets/throw observed: ____

Phase 5 done [] Re-run suite: .\env\Scripts\python.exe -m pytest tests -q -> ____ passed (expect **233**) []

PHASE 6 - Phase-2 geometry survey (R4 + vertices)

Measure the array as built and write measured values into config.yaml array: **Sensor heights** (acoustic-centre above floor; must agree within +/-3 mm - R4):

Pod	height [m]
S1	_____
S2	_____
S3	_____

Max - min = ____ m (must be ≤ 0.003). -> set S_height_m = mean ____ m. If they do NOT agree within +/-3 mm, the equal-height 2x2 reduction is invalid - re-level the pods.

Pairwise vertex distances (each ~ side s = 1.000 m):

Pair	distance [m]
S1-S2	_____
S2-S3	_____
S3-S1	_____

If materially off the ideal symmetric layout, update S1_xz/S2_xz/S3_xz from the surveyed geometry (keep the sec. 0.1 frame: +x toward S1, origin at centroid). Otherwise the ideal values stand. Record all three heights + three distances in config.yaml comments. Phase 6 done []

PHASE 7 - Final pre-throw verification

#	Check	Done
7.1	Sketch 05 flashed; verify_timing.py --port COM5 all PASS	[]
7.2	config.yaml updated: sigma_pos_m, beam_half_angle_deg, S_height_m, vertices	[]
7.3	Offset subtraction added at acquisition layer iff 4D flagged SIGNIFICANT	[]
7.4	pytest tests -q still green after config edits (expect 233)	[]
7.5	One dry-run live throw end to end gives a sane (r, theta) and writes a full session dir	[]
7.6	report.html (per-throw) + data\sessions\campaign.html were auto-rendered by the run (v2: terminal mode renders both - no --web needed)	[]
7.7	--plot produced trajectory_fit.png + range_vs_time.png for that throw	[]

Dry-run live throw:

```
.\env\Scripts\python.exe scripts\run_session.py --port COM5 --plot
```

Enter temperature when prompted; throw the ball over the array. At CLOSING it prints r = ... m, theta = ... deg, sigma_r = ... m, sigma_theta = ... deg, then the **G1 ground-truth prompt** (v2 - the old two-tape L_centroid/L_S1 prompt no longer exists):

#	G1 ground-truth flow - the terminal prompt (CLAUDE.md 5.5)
1	The pipeline RECOMMENDS the two sensors nearest the predicted landing: 'recommend references: centroid + Sx, Sy ... accept [Enter] / override [pick two of S1 S2 S3]'. Press Enter to accept, or type e.g. 'S1 S3'.
2	Measure THREE tape distances from the first-contact mark: L_centroid, then L_<first sensor>, L_<second sensor> (prompts relabel to the chosen pair). Empty input at any prompt = skip; add later with scripts\append_ground_truth.py <session_id> or scripts\serve_report.py --pending.
3	The solve is unique (no mirror-side question). An integrity warning (residual / condition number) means a mis-read tape or wrong sensor logged - re-measure; it does not block the save.

If the throw is too short/flat (v2): the program does NOT crash - it prints 'no prediction: ...' with the actionable reason (throw slower/loftier), keeps the session dir reproducible, and renders a FAILURE-state report.html that still shows the per-sensor range-vs-time panel for diagnosis. That is expected behaviour, not a fault. **Optional:** add --web to enter ground truth in the browser instead of the terminal (same solve, atomic write).

Dry-run result: r = ____ m, theta = ____ deg, triplets used ____ Phase 7 PASS []

-> BEGIN THE CAMPAIGN

You are now cleared for real throws (sec. 11 / D3): **15-20 ground-truthed throws, extend to 30 if time permits.** Vary heading deliberately (+/-60 deg about +x, off the symmetry axes). Per throw:

#	Step
1	run_session.py --port COM5 -> throw -> ground truth via the G1 prompt above (or add later: append_ground_truth.py <session_id>, or serve_report.py --pending for the browser). report.html + the live campaign.html refresh automatically after every throw and GT entry.
2	After the set: calibrate_bias.py --sessions data\sessions* -> fit/validate the drag model. The demo/ subtree is skipped automatically (session predicate) - the glob is safe. READ THE POWER NOTE: a null at 15-20 throws is 'drag below noise floor', not failure.
3	analyze_campaign.py --sessions data\sessions* -> figures + stats under campaign_report\.

Keep every raw_serial.log (no git - the tree is the only copy; zip the whole data\sessions\ tree when sharing, see README.md). A null drag result is a valid physics finding, not insufficient data.